

Quantum-Resistant Blockchain Framework for Secure AI Model Distribution and Provenance DA2

A Hybrid Ledger Architecture Combining NIST FIPS 204 (ML-DSA), Hyperledger Fabric, Ethereum Anchoring and IPFS Content Addressing

~Madhusudhan 25BCE1176

~Mahathi Shri G 25BCE5747

Abstract — AI model distribution pipelines authenticate artefacts using security mechanisms which has RSA and ECDSA signatures. But, the security of such signatures will break under the influence of Shor's algorithm. To make it worse, because blockchain stores the signatures permanently, a CRQC (Cryptographically Relevant Quantum Computer) at the end of this century will be able to use historical signatures to forge fake provenance. We propose a structure that enables a quantum-safe provenance system, consisting of NIST FIPS 204 ML-DSA-65 signatures, a Hyperledger Fabric + Ethereum combination, and an IPFS content-addressed storage. Three core ideas set it apart from traditional provenance systems: (1) Enforce signatures within chaincode Fabric does not just store keys passively on chain. Signature enforcement happens while transactions take place on-chain inside the chaincode of the ledger network. >(2) Per-tensor Merkle hashing if any change is made to a specific tensor at a layer, Merkle tree enables localization of the tampering. That way, it is also possible to check multi-gigabit models' verification through streaming. The paper acknowledges, instead of hiding, the quantum vulnerability of the system that remains after anchoring: the Ethereum anchoring transaction is signed with secp256k1

1. Introduction

1.1 Background and Motivation

Machine learning models, which previously were monolithic, black-box systems, have now been decomposed into multiple components and pieces that have very little dependency on each other. Foundation models are the large models that are not modified by the developers. The developers take these models and fine-tune or customize them per the requirements of their applications. Edge-deployed vision networks or models that are deployed only on device are one part, and another part are the models that are deployed in cloud which is continuously moved between different organisations, cloud regions and devices such that the networks through which models transit are partially or not controlled at all by parties who are directly or indirectly involved. These new ways of handling machine learning models pose challenges to federated machine learning and multi-tenant MLOps pipeline setups. These setups have to rely on weight updates and hyperparameter configurations being transmitted through these trust boundaries between different entities.

Distributed ledger technologies, like blockchains, help in solving the trust part of the problem. It is a shared ledger with the ability to add entries, a record of model hash digests, metadata tied to the model authorship, and model versions. This way, a model's entire history and provenance are captured in the audit trail without requiring any central entity to be the registry operator. Still, the protection of a ledger is only as good as the system that signs the transactions in it. Many blockchain technologies currently authenticate transactions using ECDSA signatures on secp256k1 or X.509 certificates backed by RSA. In both, the mathematical problems are essentially the reduction of the factorisations in case of integers and the discrete logarithm problem on elliptic curves, which the elliptic-curve group operations are based on. Shor showed in his research in 1994 that a large quantum computer can solve both of these mathematical problems in polynomial time and because of this render both classical signature schemes totally ineffectual[5]

1.2 The Immutability Paradox

In a world where quantum computing (QC) poses a threat to classical cryptographic systems, that property which distinguishes blockchain technology most will become its chief weakness. Suppose that a model hasher creates a public key from the signature that he also signs a model hash with today's ECDSA. That signature, as well as the public key which is the signature verification key, are written permanently onto thousands of replicated nodes. Then a CRQC (code-breaking quantum computer) comes along, expert predictions are currently in the 2030, 2040 timeframe, and an adversary takes the historic public key from the ledger, finds out the private key, and makes use of it to forge signatures on models that have been maliciously poisoned or are backdoored, which will be perceived by the system as being before the time the chain became vulnerable. As far as the chain is concerned, it is impossible to rewrite that history without causing a consensus-breakage. So, the entire chain of trust gets reverted, and all the trust is lost, going back in time.

Terminology note. Unlike HNDL where keys and messages are stored now, then decrypted later, such an attack is not limited to a single target or even just one communication, and does not rely on decryption of messages but rather their forging. Because of this, this particular kind of attack, where a key is found retrospectively and then a signature is made to appear legitimate, is quite a different thing altogether. Whereas the former is a confidentiality attack in which an enemy stores a ciphertext today and deciphers it once the key-exchange primitive has been cracked, the latter is essentially about forged signatures produced using the private key that has just been recovered. Plainly then that different kinds of solutions are required for each attack, viz. ML-KEM for the first type of attack and ML-DSA for the second kind of attack. Both types of attacks are dealt with in this paper - first ML-KEM in Section 3.2 and ML-DSA in Section 3.5).

1.3 Problem Statement

Right now, the way different AI models are disseminated has three fundamental vulnerabilities:

First of all, the whole problem of cryptographic obsolescence. At the moment, one signs authorship via a digital signature to prove one's identity, and the signature itself is supposed to be unbreakable, so that if it gets broken one knows it has been forged. Quantum key extraction technology makes it possible to recover signatures even from documents that were encrypted with RSA and ECDSA schemes, so one cannot say that the integrity of the information has been always been maintained from the very first day because once the integrity of the ledger storage gets compromised the previous integrity will also be compromised retroactively (from later to earlier).

Second, there is centralisation of trust. That is, model registries are run via centralised hubs and PKI certificate authorities that could be hacked; be pressured through legal means or have their configurations messed up. But, removing the certificate authority does not mean one can get rid of the basic problem of how to bind the public crypto key identity to that of a real-world organisation/company, the issue that is almost all times left unanswered by the decentraliser propositions.

Third is the lack of built-in, quantum-safe, provenance capabilities. Post-quantum signature schemes, blockchain provenance systems and MLOps lineage tooling are all available in a mature form. Yet no known solution integrates all the three around the unique set of constraints for AI artefacts: payloads with multi-giga-bytes of data, derivative works that are done very often (fine-tunes and LoRA adapters), and supply-chain attacks that are based on deserialisation.

1.4 Objectives

- **O1 — Integrated design.** Compose FIPS 204 ML-DSA-65, a Fabric/Ethereum dual-layer ledger and IPFS content addressing into a deployable architecture with a working reference implementation.
- **O2 — Enforced, not advertised, quantum safety.** Verify post-quantum signatures as an endorsement condition inside Fabric chaincode, so an invalid signature cannot be committed — rather than merely storing the public key on chain for optional client-side checking.
- **O3 — AI-specific tamper localisation.** Replace flat whole-file hashing with a per-tensor Merkle tree, enabling single layer tamper identification, streaming verification without full download, and cheap delta-signing of fine-tuned derivatives.
- **O4 — Honest residual-risk accounting.** Document the boundary of the quantum guarantee, including the secp256k1 dependency of the Ethereum anchor transaction and the non-persistence of unpinned IPFS content.

1.5 Scope

In scope: Fabric v2.5 network provisioning; Go chaincode with embedded ML-DSA verification; Solidity anchor contract deployed to a local Hardhat chain and the Sepolia testnet; React + ethers.js frontend; Kubo IPFS node; per-tensor Merkle hasher; unit and fault-injection test suites; gas and latency benchmarks.

Out of scope: Ethereum mainnet deployment with real value; FPGA/ASIC acceleration of lattice polynomial arithmetic; physical quantum hardware benchmarking; zero-knowledge proof circuits for on-chain ML-DSA verification (identified as future work in Section 3.10).

2. Literature Survey

2.1 Blockchain for AI Governance and Ownership

Blockchain in the management of AI and property rights is one of the most obvious applications of blockchain technology, bringing with it a whole new set of benefits. Consider what comes next specific applications:

On-platform AI; providing governance rights, procedural rights, and management information. Monetizing ownership and management of AI products. This draws on the advantages of blockchain technology for IT applications and in applications with "copy and paste" features.

Battah et al. [9] deploy smart contracts and NTFs to indicate ownership, licensing and access rights to trained models, showing that state-machines in a conventional contract are sufficient to specify a model governance. Ural and Yoshigoe [8] review blockchain-enhanced machine learning in general, focusing on incentive mechanisms and design, decentralised training markets and provenance registers. Both approaches use classical signature schemes throughout: their security statements are against dishonest participants, never against an adversary algorithmically next in a quantum universe, so their guarantees extend forward to the same fractional moment in time as secp256k1

2.2 Provenance and Lineage in MLOps

Mitchell et al. [10] proposed Model Cards, a tabulated documentation of a model's target use, training data and measure of performance, and the pattern has now become accepted. Kreuzberger et al. [15] formalise the MLOps architecture within which artefact tracking tools like MLflow and Kubeflow Hypervent set up. These show a very detailed operational lineage but house it in mutable centralised databases: an administrator or a colluding service account can rewrite history publicly visible none of them offers an unforgeable cryptographic non-repudiation guarantee that affects insider collusion.

2.3 Federated Learning Security

Kairouz et al [18] enumerate the open problems in federated learning, with most of the literature focusing on privacy-preserving aggregation (differential privacy, secure multi-party computation & homomorphic approaches) applied to gradient updates. The attack model is severely training-phase intensive. After a global model has been aggregated, the subsequent dissemination to edge devices is viewed as an implementation concern and is largely handled as assumed safe, exactly what this system is interested in.

2.4 Post-Quantum Cryptography and Its Integration into Ledgers

In August 2024 NIST rolled out final FIPS 204, approving ML-DSA (the standardized form of CRYSTALS-Dilithium [3]) as the main general-use post-quantum signature scheme plus FIPS 203 (ML-KEM) for key encapsulation [1], [2]. The scheme's security level is based on the hardness of ModuleLWE and ModuleSIS, two hard lattice problems for which no polynomial-time quantum algorithm is known [14], and it employs the FiatShamir-with-aborts rejection-sampling method originally developed by Lyubashevsky for trapdoor-free lattice signatures [4] in its signing algorithm. NIST SP 800-208 [13] is sometimes used as a citation in this area but it covers stateful hash-based schemes, namely LMS and XMSS, and does not apply to ML-DSA; the FIPS 204 paper is the relevant compliance reference.

We find Fernandez-Carames and Fraga-Lamas [7] to be the most direct precedent, including post-quantum cryptography evaluation within blockchain and listing the signature size and throughput costs that lattice schemes add to the transaction format. Their review is yet, crypto currency-centric and does not quantify the cost of heavy off-chain payloads, MLOps workflows or AI artefact structures.

2.5 Comparative Analysis Matrix

Reference	Core Contribution	Ledger	PQC	Principal Limitation
Ducas et al. [3]; NIST FIPS 204 [1]	ML-DSA lattice signature scheme and its standardisation	None	Yes	Pure primitive; no distributed state layer, no artefact-management context
Lyubashevsky [4]	Trapdoor-free lattice signatures; rejection sampling	None	Yes	Theoretical foundation only
Benet [6]	IPFS content-addressed P2P file system	DHT / libp2p	No	Storage only; no governance, no persistence guarantee without pinning
Buterin [12]	Programmable public ledger	Ethereum	No (secp256k1)	ECDSA transaction signing; gas cost prohibits large payloads
Hyperledger [11]	Permissioned ledger with MSP based RBAC	Fabric v2.5	No (X.509 RSA)	Classical PKI trust root; no public auditability

Battah et al. [9]	NFT-based model ownership and access control	Ethereum	No	Quantum-vulnerable ownership records; no integrity binding to weights
Ural & Yoshigoe [8]	Survey of blockchain-enhanced ML	Mixed	No	Classical threat model throughout
Mitchell et al. [10]	Model Cards for structured reporting	None	No	Documentation convention; no cryptographic binding
Fernández-Caramès & Fraga-Lamas [7]	Review of post-quantum blockchain cryptography	Mixed	Yes	Cryptocurrency-focused; no large-payload or AI artefact treatment
This work	Enforced ML-DSA verification + per-tensor Merkle provenance	Hybrid (Fabric + Eth)	Yes (ML-DSA-65)	Anchor transaction remains secp256k1-signed (Section 3.10)

Table 2.1 — Comparative positioning against representative prior work.

2.6 Identified Research Gap

The literature partitions cleanly along three axes that no single work spans. Blockchain-for-AI research assumes classical cryptography. Post-quantum cryptography research treats signatures as network protocol components, benchmarking TLS handshakes rather than ledger state. MLOps provenance tooling assumes a trusted central store. The gap this work addresses is therefore not “use PQC on a blockchain” — which [7] already anticipates — but the harder composition problem: enforcing post-quantum authorship inside a permissioned ledger’s endorsement policy, at a payload scale where the artefact cannot go on-chain, with verification granularity matched to the internal structure of neural network files.

3. Methodology

3.1 System Architecture

The framework comprises six components arranged in a publish-then-verify pipeline. Fabric performs high-frequency MLOps state transitions with signature enforcement and no gas cost; Ethereum receives periodic Merkle-root commitments that provide public, tamper-evident proof of existence without exposing proprietary metadata.

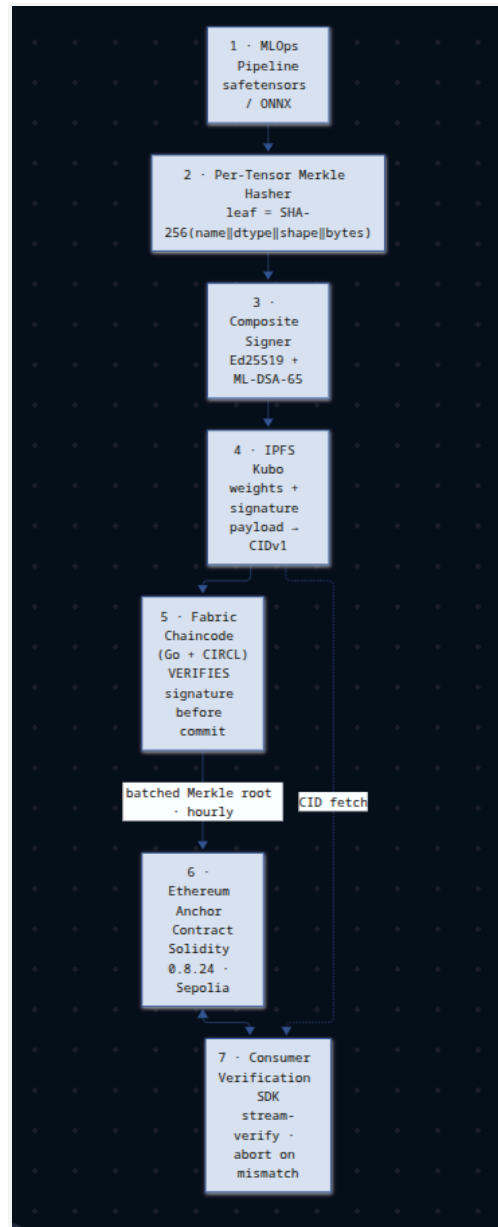


Figure 3.1 — End-to-end architecture. Verification occurs at component [5], not only at [7].

3.2 Cryptographic Design

The framework selects ML-DSA-65 (NIST Security Category 3, comparable to AES-192 key search, i.e. approximately 2^{192} classical effort). Parameter sizes are taken from FIPS 204 as standardised, which differ from the widely reproduced Round 3 Dilithium figures; the discrepancy is noted because several published surveys still quote the older values.

Parameter Set	NIST Level	Public Key (B)	Private Key (B)	Signature (B)	Round-3 signature (B)
ML-DSA-44	Category 2	1,312	2,560	2,420	2,420
ML-DSA-65 (selected)	Category 3	1,952	4,032	3,309	3,293
ML-DSA-87	Category 5	2,592	4,896	4,627	4,595

Table 3.1 — FIPS 204 parameter sets. The final column shows the superseded Round-3 Dilithium values for comparison.

Composite signatures. A signer first generates a composite signature as a concatenation of Ed25519 and ML-DSA-65 signatures, the verifier then checks that both signatures are valid. When it comes to IETF migration scenario, this method gives protection against unforeseen breaking developments in crypto analysis of lattice-based systems during a transition phase but with a price of the 64 increased bytes. A signer can only produce a signature after the signer has created and signed with a domain-separated context string.

```
μ = SHAKE256( "QRB-v1" || chainID || modelId || version || merkleRoot || pkHash )
```

which binds every signature to a single chain, model identifier and version, preventing cross-version and cross-chain replay.

Confidentiality. When proprietary model weights are deployed on the HNDLs, the data is protected via a content encryption key used to encrypt the payload using AES-256-GCM. To wrap the said content encryption key, one would use ML-KEM-768 (FIPS 203)[2] instead of ECDH. The threat vector that was named as "HNDL vector" in our Section 1.2 is still present in the system, if this level of protection is not used regardless of the strength of the signature.

3.3 Per-Tensor Merkle Hashing

SHA-256 of one serialised file is flat and will only say something was tampered with without a clue where it was, it needs the full multigigabyte file downloaded, which verification is only after that it starts, and one change like a fine-tune that modifies a handful of adapter matrices will force the complete re-signature. The structure parses the artefact as a safetensors container without having the full download first, then hashes each named tensor separately, sorts the tensor names to ensure consistency, and builds a binary Merkle tree, the root of which will be signed:

```
leaf_i = SHA-256( name_i || dtype_i || shape_i || tensor_bytes_i )
root = MerkleRoot( sort_by_name( leaf_0 ... leaf_n ) )
H_meta = SHA-256( model_card || dataset_hash || eval_metrics )
signed_μ = SHAKE256( context || root || H_meta )
```

One of the differences between flat hash and Merkle hash, for properties, is that there are three benefits which a flat hash cannot do. Tamper localisation: in case of a failed inclusion proof, you can immediately identify the layer in the tree that has been altered. Streaming verification: a user can check each tensor as it is arriving and in case of a mismatch at the first attempt, the user will stop the verification process. Delta provenance: a LoRA adapter re-signs only the changed leaves and records a pointer to its parent root, resulting in a lineage DAG instead of a flat list of unrelated artifacts.

Deserialisation defence. The consumer application software development kit (SDK) performs the validation first and then carries out the loading. By far, the most common real-world supply-chain attack at the moment targeting AI isn't through quantum computing but running arbitrary code by exploiting a deserialization feature in pickle through torch. load; in such a case, a verified signature after loading is of no value. So, the SDK outright rejects pickle-backup formats and only allows safetensors and ONNX.

3.4 Storage Layer

Model binaries are pinned to a Kubo IPFS cluster, which chunks the file into a UnixFS Merkle DAG and returns a CIDv1. It is worth stating the content identifier precisely, since it is commonly misreported:

```
CID = CIDv1( codec = dag-pb, multihash = SHA-256( UnixFS DAG root ) )
```

For any file exceeding the 256 KiB default chunk size, this root differs from the SHA-256 digest of the file bytes, so the Merkle root of Section 3.3 is computed and recorded separately. IPFS content addressing guarantees integrity but not persistence: unpinned blocks are garbage-collected. The reference deployment therefore maintains three pinning nodes plus a Filecoin storage deal for artefacts under long-term retention policy, and the residual availability risk is rated MEDIUM in Table 3.3.

3.5 Smart Contract and Chaincode Design

Responsibility is shared among participants. Fabric chaincode checks the signature during endorsement using the Cloudflare CIRCL library [16], providing Go native ML-DSA without the need for cgo bindings. A signature check of 3,309 bytes taking 0.3 ms can be done as a simple endorsement condition in Fabric, with no gas metering, so the invalid signature is rejected before it is committed. Whereas the same check would be very expensive for EVM opcodes, Ethereum just stores the commitments.

Three defects of the anchor contract were removed when the contract design was revised: a on-chain public key of 1,952 bytes needing about 1.2 M gas to store, an indexed string event parameter (Solidity stores a string as a keccak hash, so you can't read the model ID from logs), and a presence check that halted genuine version updates.

```
struct Anchor {
    bytes32 merkleRoot; // per-tensor Merkle root (Section 3.3)
    bytes32 cidDigest; // CIDv1 multihash digest, not a string
    bytes32 pkHash; // keccak256(ML-DSA pk); full key lives on IPFS
    bytes32 parentRoot; // lineage pointer — 0x0 for a base model
```

```

uint64 version;
uint64 timestamp;
bool revoked;
}
mapping(bytes32 => mapping(uint64 => Anchor)) private registry; // id → version

event ModelAnchored(bytes32 indexed modelIdHash, string modelId,
uint64 version, bytes32 merkleRoot, bytes32 cidDigest);
event ModelRevoked(bytes32 indexed modelIdHash, uint64 version, string reason);

function anchorModel(string calldata modelId, uint64 version, bytes32 merkleRoot, bytes32 cidDigest, bytes32
pkHash, bytes32 parentRoot)
external onlyPublisher {
bytes32 k = keccak256(bytes(modelId));
require(registry[k][version].merkleRoot == 0, "version already anchored");
require(version == 1 || registry[k][version-1].merkleRoot != 0, "gap in lineage"); /* ... store, emit ... */
}

```

Publisher management is placed behind a 2-of-3 Gnosis Safe multisignature wallet, and revocation is a first-class operation: a compromised signing key can be retired and the affected versions flagged without rewriting history.

3.6 Network Setup and Frontend Integration

Layer	Configuration	Purpose / Verification Artefact
Fabric	v2.5 LTS, 2 organisations × 2 peers, 3-node Raft orderer, channel "mlops", Go chaincode with CIRCL	Signature enforcement; endorsement policy AND(Org1MSP.peer, Org2MSP.peer). Evidence: peer logs showing endorsement rejection on a corrupted signature
Ethereum	Hardhat local node (chainId 31337) for development; Sepolia testnet for public anchoring; Solidity 0.8.24	Public proof of existence. Evidence: deployed contract address and anchoring transaction hashes
IPFS	Kubo v0.28, 3-node private swarm, Pinata fallback pinning	Payload storage. Evidence: CIDv1 values and pin status output
Crypto	liboqs / python-oqs for the signer CLI [17]; CIRCL for chaincode; PyNaCl for Ed25519	Keypair generation and composite signing
Frontend	React 18 + ethers.js v6 + Fabric Gateway SDK; MetaMask for anchor transactions	Upload → hash → sign → pin → submit → verify workflow with live status
Consumer SDK	Python package: fetch CID, stream-verify tensors, check ML-DSA signature, confirm anchor inclusion proof	Refuses to return a model object if any check fails

Table 3.2 — Deployment environment and the evidence each layer produces for the functional demonstration.

The frontend brings four different user journeys: registration of a model, traversal of model lineage ("rendering" a parent-pointer DAG), artefact download and verification with a view of per-tensor pass/fail, and the administration of the rights publisher and revocation. Interactions related to the Fabric component are done via the Gateway SDK through gRPC; Ethereum anchoring is done through the browser via MetaMask signing.

3.7 Evaluation Methodology

The following measurements represent the results part of the functional demonstration.

- The first one is signing and verification latency for ML-DSA-65 versus ECDSA-secp256k1 and Ed25519, measured separately from artefact I/O.
- The second is the per-tensor Merkle construction throughput measured against model size (10 MB, 100 MB, 1 GB, 7 GB), benchmarked against flat SHA 256, to quantify the overhead of tamper localisation.
- Third is gas consumption of anchorModel() and revokeModel(), and the amortised cost per model when N transitions are batched and processed with one Merkle root.
- Fourth is Fabric endorsement latency and throughput (TPS) with signature verification enabled versus the cost of enforcement without verification.

- Fifth is fault injection: single-bit flips in a randomly selected tensor, tensor reordering(cid), CID substitution, replay of a prior version and registration by an unauthorised publisher. Expected outcome is detection in every case with a description in a per tensor test also stating the layer affected.

3.8 Threat Model and Residual Risk

STRIDE Class	Threat	Control	Residual
Spoofing	Adversary publishes a malicious model under a trusted vendor identity	Composite Ed25519 + ML-DSA-65 verified during Fabric endorsement	LOW
Tampering	Weight modification in transit or at an IPFS gateway	Per-tensor Merkle root; streaming verification with layer-level localisation	LOW
Repudiation	Vendor denies authoring a model that later failed	Non-repudiable ML-DSA signature anchored to Ethereum; note block.timestamp is miner-influenceable by seconds	LOW
Information disclosure	Leakage of proprietary architecture through public metadata	Weights encrypted under ML-KEM-768- wrapped AES-256-GCM; metadata confined to Fabric private data collections	LOW
Denial of service	Unpinned IPFS content garbage-collected; registry spam	Triple pinning plus Filecoin deal; onlyPublisher modifier and Fabric MSP gatekeeping	MEDIUM
Elevation of privilege	CRQC recovers the publisher's secp256k1 key and anchors forged records	2-of-3 multisig on publisher management; Fabric-side ML-DSA check still fails, so forged anchors are detectable but not preventable	MEDIUM

Table 3.3 — STRIDE assessment. Two risks are deliberately rated MEDIUM rather than mitigated to LOW.

Stated limitation. The Ethereum anchoring transaction is authorised by *msg.sender* and therefore signed with secp256k1. A quantum adversary who recovers a publisher's externally owned account key can write anchors, even though they cannot forge the ML-DSA signature those anchors reference — meaning forged anchors fail verification but can still be committed. Closing this gap requires an ERC-4337 smart account with an ML-DSA verifier module, or a zk-SNARK proving correct off-chain lattice verification so the EVM checks only a succinct proof. Both are identified as the primary direction for continued work; neither is claimed as implemented here.

3.9 Implementation Schedule

Week	Phase	Deliverable
1	Survey, threat modelling, parameter selection	Literature matrix; STRIDE table; ML-DSA-65 justification
2	Fabric network and Hardhat environment provisioning	Running 2-org channel; local chain with funded accounts
3	Chaincode and Solidity contract development	CIRCL-based verification chaincode; anchor contract with passing unit tests
4	Signer CLI and per-tensor Merkle hasher	Composite signing tool; Merkle hasher validated on a 1 GB model
5	IPFS pipeline, frontend integration, end-to-end run	React UI completing upload→anchor→verify; fault-injection results
6	Benchmarking, Sepolia deployment, documentation	Gas and latency tables; public contract address; final report

